

Book Store

Online Book Shopping System Using MERN Stack

Project Report

Prepared by: Merugu Neeharika
Full Stack Developer

Contents

1. Introduction

- 1.1 Project Title
- 1.2 Team Members
- 1.3 Introduction

2. Project Overview

- 2.1 Purpose
- 2.2 Objectives
- 2.3 Key Features

3. Architecture

- 3.1 System Architecture
- 3.2 Frontend Architecture
- 3.3 Backend Architecture
- 3.4 Database Architecture

4. Setup Instructions

- 4.1 Prerequisites
- 4.2 Installation

5. Folder Structure

- 5.1 Client Folder Structure
- 5.2 Server Folder Structure

6. Running the Application

- 6.1 Starting the Backend Server
- 6.2 Starting the Frontend
- 6.3 Application Workflow

7. API Documentation

- 7.1 Introduction
- 7.2 Authentication APIs
- 7.3 Book APIs
- 7.4 Wishlist APIs
- 7.5 Shopping Cart APIs
- 7.6 Review APIs
- 7.7 Order APIs

8. Authentication

- 8.1 Authentication Mechanism
- 8.2 Authorization
- 8.3 Authentication Workflow
- 8.4 Advantages of JWT Authentication

9. User Interface

- 9.1 Introduction

10. Testing

- 10.1 Testing Strategy
- 10.2 Test Cases
- 10.3 Testing Tools

11. Screenshots

- 11.1 Home Page
- 11.2 Login Page
- 11.3 Registration Page
- 11.4 Book Catalog
- 11.5 Book Details
- 11.6 Wishlist
- 11.7 Shopping Cart
- 11.8 Orders
- 11.9 Admin Dashboard

12. Known Issues

13. Future Enhancements

14. Conclusion

Chapter 1

Introduction

1.1 Project Title

Book Store: Online Book Shopping System Using MERN Stack

1.2 Team Members

S.No	Name	Role
1	Merugu Neeharika	Full Stack Developer

1.3 Introduction

The Book Store is a web-based application developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js) to simplify the process of buying books online. The application provides an intuitive interface where users can browse books, search by title or category, view detailed information, add books to a wishlist or shopping cart, and place orders.

The system also provides authentication and authorization using JSON Web Tokens (JWT), ensuring that only authenticated users can access protected resources. Passwords are securely encrypted using the bcrypt library before being stored in the database. Administrators have exclusive access to manage books, categories, users, and customer orders through an admin dashboard.

Chapter 2

Project Overview

2.1 Purpose

The primary purpose of this project is to provide an efficient online platform for purchasing books. Traditional bookstores often require customers to visit physical stores, making it difficult to browse a large collection of books conveniently. The proposed system overcomes these limitations by allowing users to access the bookstore anytime and from anywhere.

The application aims to provide a secure, scalable, and user-friendly shopping experience while enabling administrators to manage inventory, orders, and customer information effectively.

2.2 Objectives

The major objectives of the project are:

- Develop a responsive online bookstore using the MERN Stack.
- Provide secure user registration and login using JWT authentication.
- Allow users to browse books based on categories.
- Implement powerful search functionality.
- Enable users to add books to their wishlist.
- Provide shopping cart functionality.
- Allow customers to place and manage orders.
- Implement reviews and ratings for books.
- Develop an admin dashboard for managing books and users.
- Maintain data securely using MongoDB.

2.3 Key Features

The major features of the Book Store application include:

- User Registration and Login
- JWT Authentication
- Password Encryption using bcrypt
- Book Management
- Category Management
- Search and Filtering
- Shopping Cart

- Wishlist
- Reviews and Ratings
- Order Management
- Admin Dashboard
- Responsive User Interface
- RESTful API Communication
- MongoDB Database Integration

Chapter 3

Architecture

3.1 System Architecture

The Book Store application follows the MERN architecture consisting of React.js for the frontend, Node.js with Express.js for the backend, and MongoDB as the database. The architecture separates the presentation layer, business logic, and data storage, making the application scalable and easy to maintain.

The frontend is responsible for rendering user interfaces and handling user interactions. Axios is used to communicate with the backend through REST APIs. The backend validates requests, performs authentication using JWT, executes business logic, and interacts with MongoDB using Mongoose.

3.2 Frontend Architecture

The frontend is developed using React.js and provides a responsive, component-based user interface. React Router is used for client-side routing, allowing seamless navigation between pages without reloading the application.

The frontend consists of the following major components:

- Home Page
- User Registration
- Login Page
- Book Listing
- Book Details
- Category Page
- Wishlist
- Shopping Cart
- Orders
- Reviews and Ratings
- User Profile
- Admin Dashboard

Axios is used to send HTTP requests to the backend APIs and receive JSON responses, ensuring smooth communication between the client and server.

3.3 Backend Architecture

The backend is developed using Node.js and Express.js. It exposes RESTful APIs that handle all business operations, including user authentication, book management, wishlist management,

shopping cart operations, reviews, and order processing.

The backend consists of multiple modules:

- Authentication Module
- User Management Module
- Book Management Module
- Category Management Module
- Wishlist Module
- Shopping Cart Module
- Review and Rating Module
- Order Management Module
- Admin Module

JWT is used to authenticate users, while bcrypt securely hashes user passwords before storing them in MongoDB.

3.4 Database Architecture

MongoDB is used as the primary NoSQL database for storing application data. Mongoose provides schema definitions and simplifies communication between Node.js and MongoDB.

The major collections include:

- Users
- Books
- Categories
- Wishlist
- Cart
- Reviews
- Orders

Relationships between collections are maintained using ObjectId references, allowing efficient retrieval of related information while ensuring database consistency.

The architecture provides high performance, modularity, scalability, and ease of maintenance, making it suitable for modern e-commerce applications.

Chapter 4

Setup Instructions

4.1 Prerequisites

Before setting up the Book Store application, ensure that the following software and tools are installed on your system.

Software	Version
Operating System	Windows / Linux / macOS
Node.js	v18 or above
MongoDB	v6.0 or above
Visual Studio Code	Latest Version
Git	Latest Version
Web Browser	Google Chrome / Edge
Postman	Latest Version

The following Node.js packages are required for successful execution of the application.

- Express.js
- React.js
- MongoDB
- Mongoose
- Axios
- JWT (jsonwebtoken)
- bcrypt
- dotenv
- cors
- nodemon

4.2 Installation

The Book Store project consists of two separate folders:

- Client (React Frontend)
- Server (Node.js Backend)

Clone the repository using Git.

```
git clone https://github.com/yourusername/bookstore.git
```

Navigate into the project directory.

```
cd bookstore
```

Install frontend dependencies.

```
cd client  
npm install
```

Install backend dependencies.

```
cd ../server  
npm install
```

Create a .env file inside the server directory and configure the following environment variables.

```
PORT=5000  
MONGO_URI=<your_mongodb_connection_string>  
JWT_SECRET=<your_secret_key>
```

After configuring the environment variables, the project is ready for execution.

Chapter 5

Folder Structure

5.1 Client Folder Structure

The client folder contains all frontend files developed using React.js.

```
client/  
  public/  
  src/  
    assets/  
    components/  
    context/  
    pages/  
    services/  
  App.jsx  
  index.js  
package.json  
node_modules/
```

5.1.1 Description

Folder	Description
components	Reusable UI components
pages	Individual application pages
context	Global state management
services	API communication using Axios
assets	Images, icons and CSS files
App.jsx	Main application component
index.js	Entry point of React application

5.2 Server Folder Structure

The backend is organized into different folders based on functionality.

```
server/  
  config/  
  controllers/  
  middleware/  
  models/  
  routes/  
  uploads/  
  utils/  
  server.js  
package.json  
node_modules/
```

5.2.1 Description

Folder	Description
config	Database configuration
controllers	Business logic implementation
middleware	Authentication and validation
models	MongoDB schemas
routes	REST API routes
uploads	Stores uploaded images
utils	Helper functions
server.js	Application entry point

Chapter 6

Running the Application

6.1 Starting the Backend Server

Open the terminal and navigate to the server directory.

```
cd server  
npm start
```

If Nodemon is configured:

```
npm run dev
```

The backend server starts on <http://localhost:5000>

6.2 Starting the Frontend

Open another terminal window and navigate to the client directory.

```
cd client  
npm start
```

The React application starts on <http://localhost:3000>

6.3 Application Workflow

The execution flow of the Book Store application is as follows.

- 1 Start the MongoDB server.
- 2 Start the backend server.
- 3 Start the React frontend.
- 4 Open the browser.
- 5 Register a new account or login.
- 6 Browse available books.
- 7 Search books using keywords.
- 8 Add books to Wishlist.
- 9 Add books to Shopping Cart.
- 10 Place an order.
- 11 Submit reviews and ratings.
- 12 Administrator manages books, users and orders.

The frontend communicates with the backend through REST APIs using Axios, while the backend performs authentication, business logic, and database operations through MongoDB and Mongoose.

Chapter 7

API Documentation

7.1 Introduction

The backend of the Book Store application exposes a set of RESTful APIs developed using Node.js and Express.js. These APIs enable communication between the React frontend and the MongoDB database. The APIs handle user authentication, book management, shopping cart operations, wishlist management, reviews, ratings, and order processing.

The APIs follow standard HTTP methods such as GET, POST, PUT, and DELETE. Data is exchanged in JSON format, making the communication efficient and lightweight.

7.2 Authentication APIs

API Endpoint	Method	Description
/api/auth/register	POST	Register a new user
/api/auth/login	POST	User login
/api/auth/profile	GET	Fetch user profile

7.3 Book APIs

API Endpoint	Method	Description
/api/books	GET	Get all books
/api/books/:id	GET	Get book details
/api/books	POST	Add a new book
/api/books/:id	PUT	Update book
/api/books/:id	DELETE	Delete book

7.4 Wishlist APIs

API Endpoint	Method	Description
/api/wishlist	GET	View wishlist
/api/wishlist	POST	Add book to wishlist
/api/wishlist/:id	DELETE	Remove book from wishlist

7.5 Shopping Cart APIs

API Endpoint	Method	Description
/api/cart	GET	View shopping cart
/api/cart	POST	Add item to cart
/api/cart/:id	PUT	Update quantity
/api/cart/:id	DELETE	Remove item from cart

7.6 Review APIs

API Endpoint	Method	Description
/api/reviews	POST	Add review
/api/reviews/:bookId	GET	Get all reviews of a book

7.7 Order APIs

API Endpoint	Method	Description
/api/orders	POST	Place an order
/api/orders	GET	View all orders
/api/orders/:id	GET	View order details

Chapter 8

Authentication

8.1 Authentication Mechanism

The Book Store application implements secure authentication using JSON Web Token (JWT). Users are required to register with a valid email address and password. During registration, passwords are encrypted using the bcrypt library before being stored in the MongoDB database.

After successful login, the server generates a JWT token, which is returned to the client. The React frontend stores the token securely and includes it in the Authorization header for all protected API requests.

Only authenticated users can access features such as placing orders, managing wishlists, adding reviews, and viewing order history.

8.2 Authorization

The application supports role-based authorization.

- **Customer**

- Browse books
- Add books to cart
- Add books to wishlist
- Place orders
- Submit reviews and ratings

- **Administrator**

- Manage books
- Manage categories
- View all users
- Manage orders
- Update inventory

8.3 Authentication Workflow

The authentication process follows these steps.

- 1 User registers using name, email, and password.
- 2 Password is encrypted using bcrypt.
- 3 User logs into the application.
- 4 Server validates user credentials.

- 5 JWT token is generated.
- 6 Token is sent to the frontend.
- 7 Frontend stores the token.
- 8 Token is included in future API requests.
- 9 Backend validates the token before granting access.

8.4 Advantages of JWT Authentication

- Secure communication between client and server.
- Stateless authentication.
- Faster request processing.
- Easy integration with REST APIs.
- Improved scalability.
- Better security through token expiration.

Chapter 9

User Interface

9.1 Introduction

The Book Store application provides a responsive and user-friendly interface developed using React.js. The user interface is designed to offer smooth navigation, attractive layouts, and an enhanced shopping experience. Bootstrap and CSS are used to create responsive web pages that adapt to different screen sizes.

The application consists of separate interfaces for customers and administrators. Customers can browse books, manage their shopping cart, wishlist, and orders, while administrators can manage books, users, and orders through the admin dashboard.

Chapter 10

Testing

10.1 Testing Strategy

The Book Store application was tested using both manual testing and API testing. Functional testing ensured that all application modules behaved according to the specified requirements.

Postman was used to test REST APIs, while Google Chrome Developer Tools were used to inspect frontend functionality.

10.2 Test Cases

S.No	Test Case	Expected Result	Status
1	User Registration	User account created successfully	Pass
2	User Login	JWT Token generated	Pass
3	Search Books	Matching books displayed	Pass
4	Add to Wishlist	Book added successfully	Pass
5	Remove Wishlist Item	Item removed	Pass
6	Add to Cart	Cart updated successfully	Pass
7	Update Quantity	Quantity modified	Pass
8	Delete Cart Item	Item removed	Pass
9	Place Order	Order placed successfully	Pass
10	Submit Review	Review stored successfully	Pass
11	View Orders	Order history displayed	Pass
12	Admin Login	Dashboard displayed	Pass
13	Add Book	Book inserted into database	Pass
14	Update Book	Book details updated	Pass
15	Delete Book	Book removed successfully	Pass

10.3 Testing Tools

The following tools were used during testing: Google Chrome, Postman, MongoDB Compass, Visual Studio Code, React Developer Tools, and Node.js Runtime.

Chapter 11

Screenshots

This chapter presents screenshots of the developed Book Store application. The screenshots demonstrate the successful implementation of all major functionalities.

11.1 Home Page

The Home Page is the landing page of the application. It displays featured books, newly added books, categories, and navigation menus. Users can browse available books without logging in.

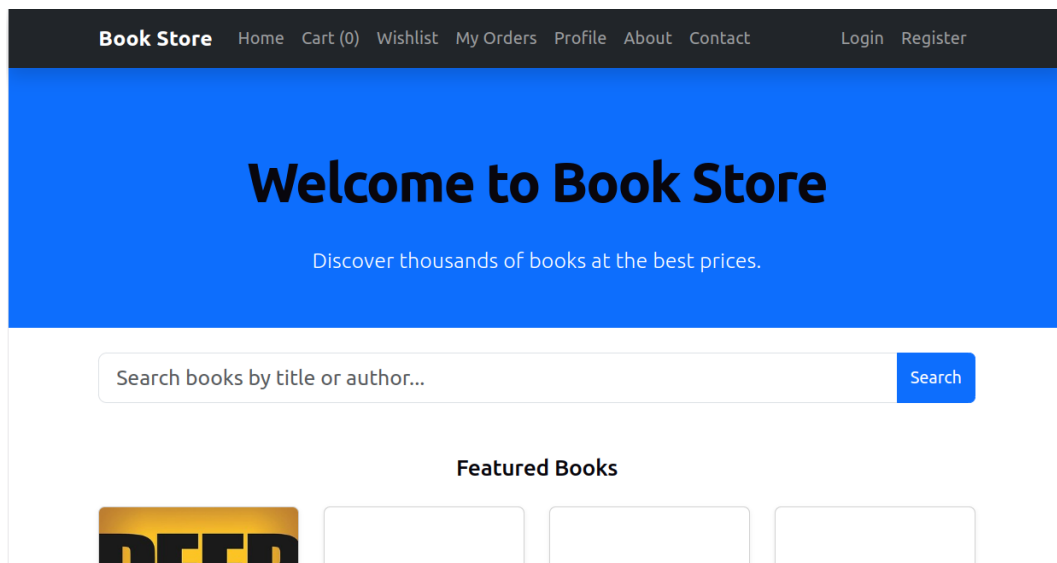


Figure 11.1: Home Page

11.2 Login Page

The Login page allows registered users to authenticate using their email address and password. JWT authentication is implemented to provide secure access to protected resources.

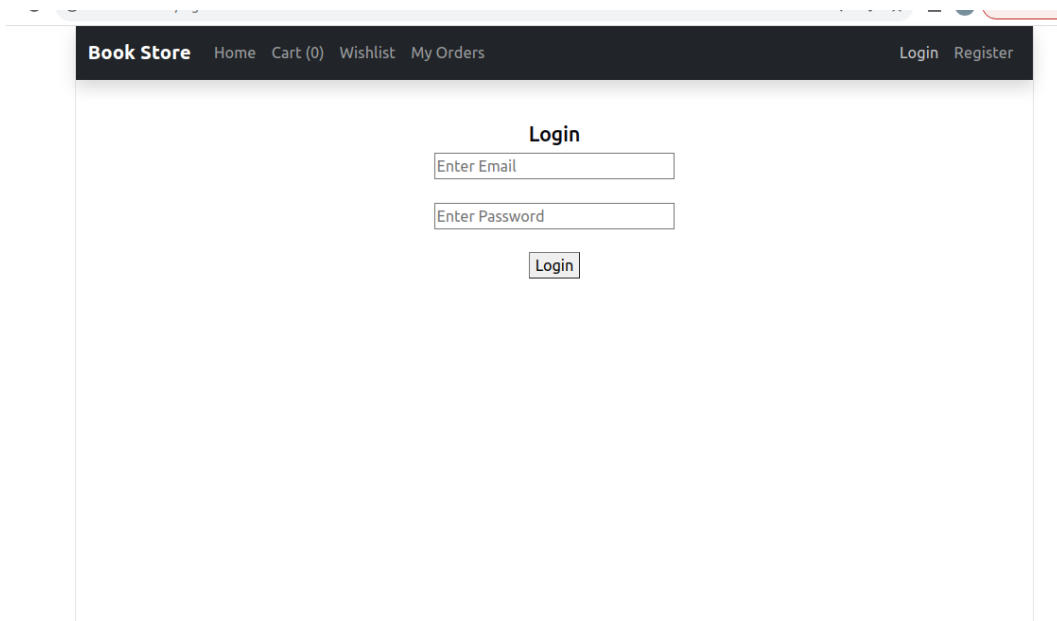


Figure 11.2: Login Page

11.3 Registration Page

New users can create an account by providing their name, email address, and password. Passwords are encrypted before being stored in the database.

Book Store Home Cart (0) Wishlist My Orders Profile About Contact Login Register

Register

Name

Email

Password

Register

Figure 11.3: Registration Page

11.4 Book Catalog

The Book Catalog page displays all available books with their title, author, price, category, and cover image. Users can search and filter books based on their preferences.

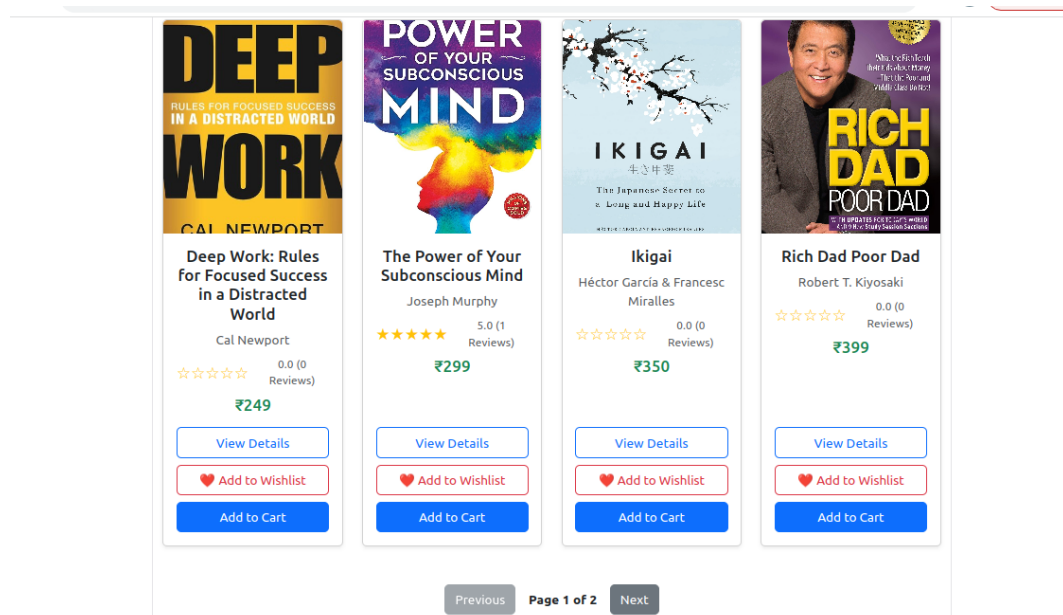


Figure 11.4: Book Catalog

11.5 Book Details

The Book Details page provides complete information about a selected book, including description, author, price, ratings, reviews, and stock availability.

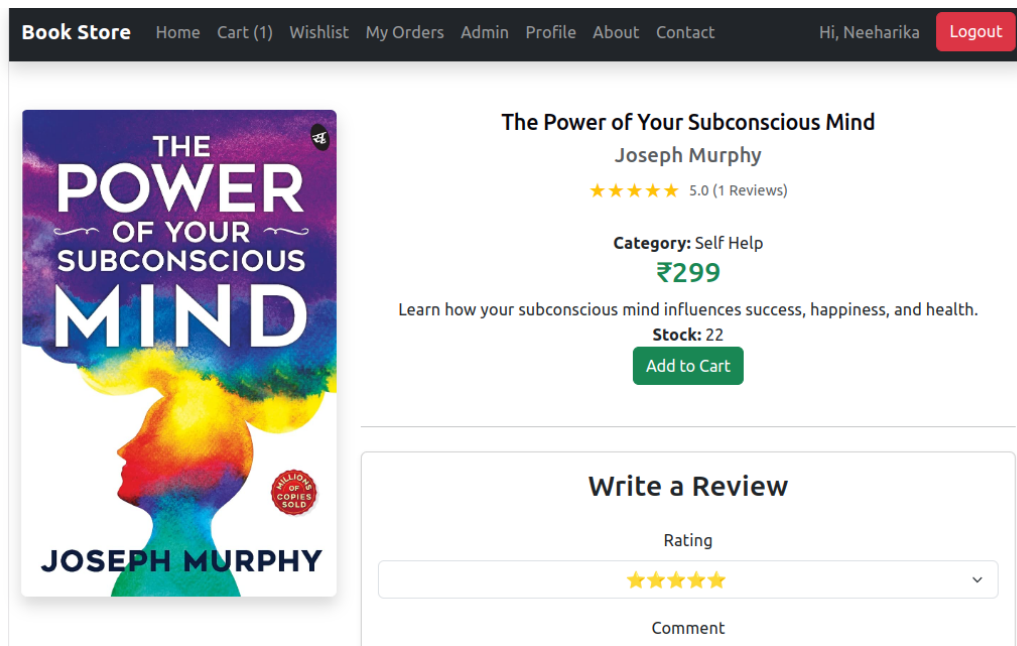


Figure 11.5: Book Details

11.6 Wishlist

Users can save their favourite books in the wishlist for future purchases. Books can be removed or moved directly to the shopping cart.

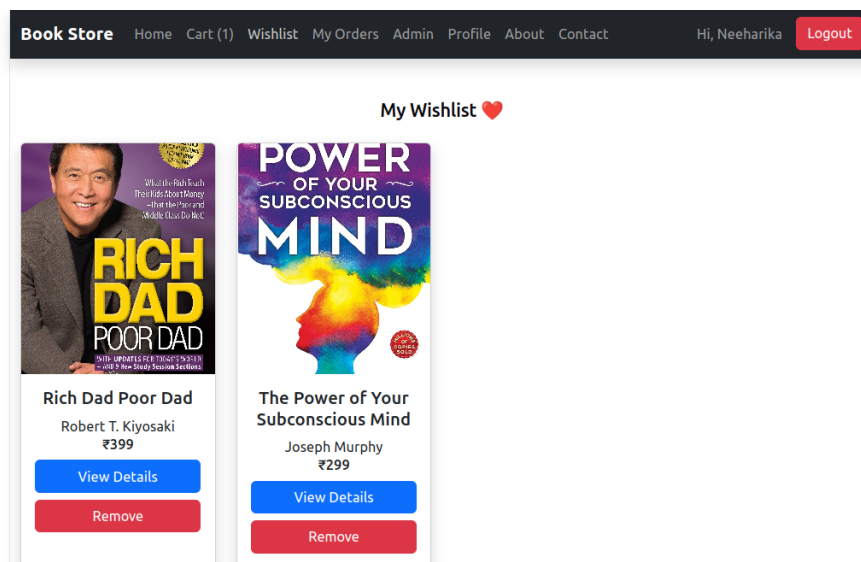


Figure 11.6: Wishlist

11.7 Shopping Cart

The Shopping Cart displays selected books along with quantity, price, and total amount. Users can update quantities or remove books before placing an order.

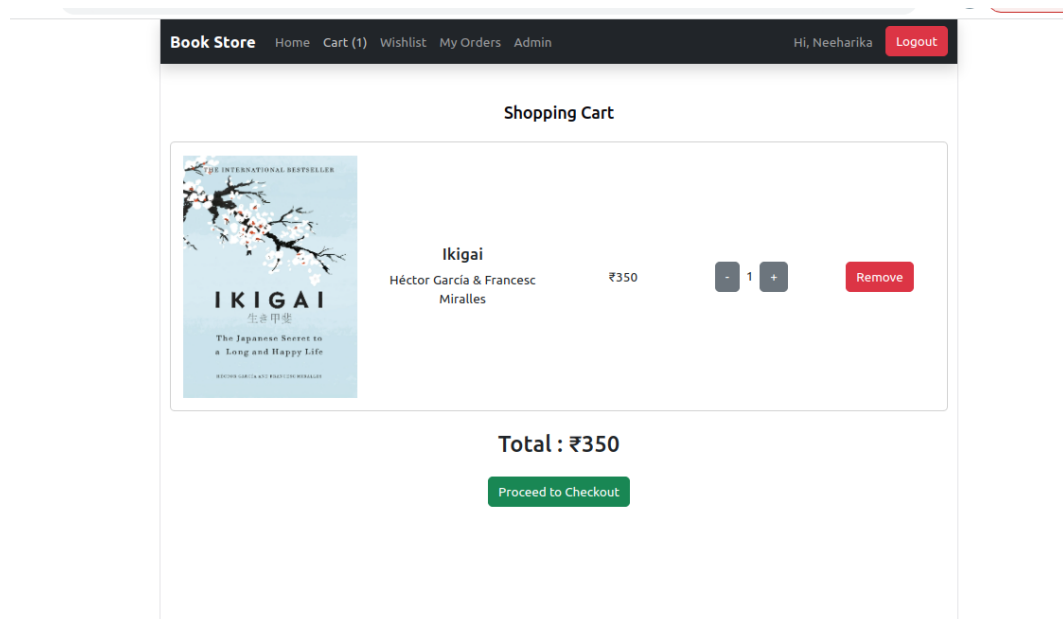


Figure 11.7: Shopping Cart

11.8 Orders

The Orders page displays all orders placed by the user along with order status and purchase history.

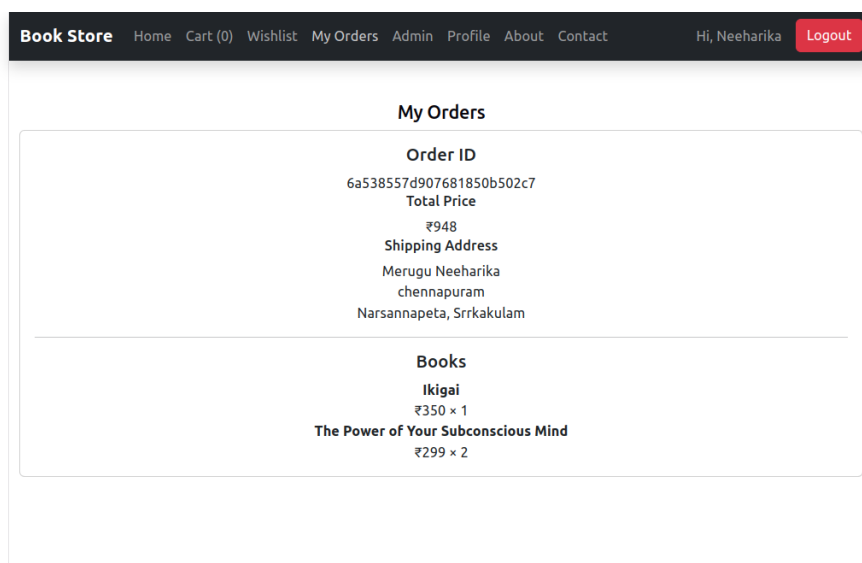


Figure 11.8: Orders

11.9 Admin Dashboard

The Admin Dashboard provides administrative features for managing books, categories, users, and customer orders. It also displays system statistics such as total users, books, and orders.

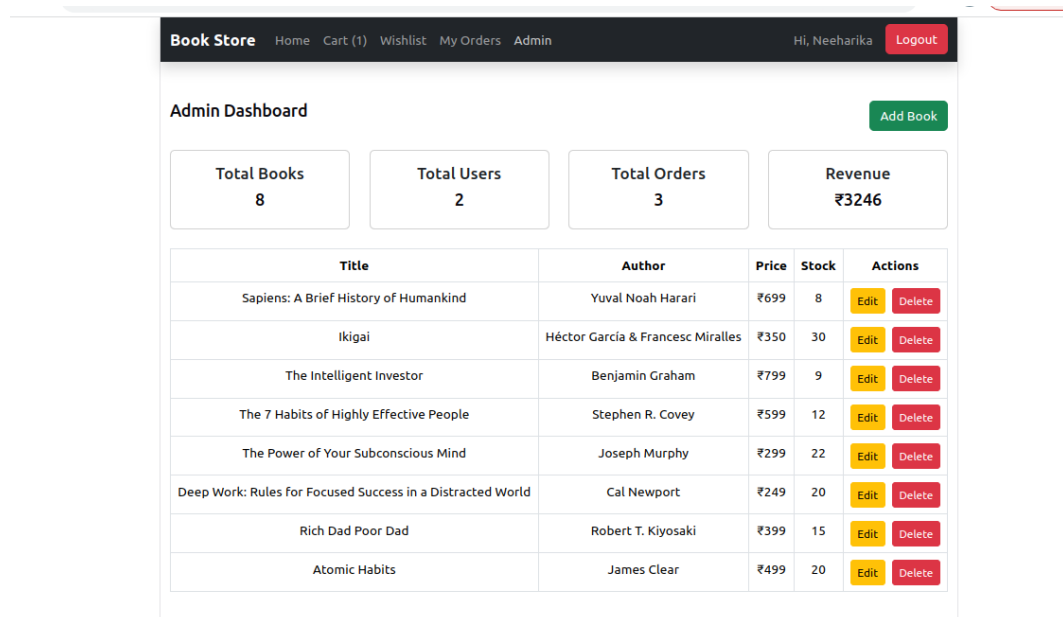


Figure 11.9: Admin Dashboard

Chapter 12

Known Issues

Although the Book Store application is fully functional, a few limitations and minor issues still exist. These issues can be addressed in future versions to improve the overall performance and user experience.

- The application currently supports only Cash on Delivery (COD). Online payment gateway integration has not yet been implemented.
- Email notifications for user registration, order confirmation, and password reset are not available.
- The recommendation system is based only on book categories and does not provide personalized recommendations.
- The application does not support multiple languages.
- Real-time order tracking is currently unavailable.
- Image optimization and lazy loading can be improved for faster page loading.
- The application is designed primarily for web browsers and does not have a dedicated mobile application.
- Advanced analytics and sales reports for administrators are limited.

The above limitations do not affect the core functionality of the system, which successfully performs user authentication, book browsing, shopping cart management, wishlist management, reviews, ratings, and order processing.

Chapter 13

Future Enhancements

The Book Store application can be extended with several advanced features to improve usability, security, and scalability.

- Integration of secure online payment gateways such as Razorpay, Stripe, or PayPal.
- AI-based recommendation system to suggest books according to user preferences and purchase history.
- Email notifications for registration, password reset, and order confirmation.
- SMS notifications for delivery updates.
- Real-time order tracking.
- Barcode and QR code support for inventory management.
- Multi-language support to serve users from different regions.
- Voice-based book search.
- Cloud deployment using AWS, Microsoft Azure, or Google Cloud Platform.
- Docker containerization for simplified deployment.
- Advanced analytics dashboard for administrators.
- Mobile application using React Native.
- Social media login using Google and Facebook authentication.
- Chatbot support for customer assistance.
- Dark mode and additional accessibility features.

These enhancements will significantly improve the overall user experience and make the application suitable for large-scale commercial deployment.

Chapter 14

Conclusion

14.1 Conclusion

The Book Store application was successfully developed using the MERN Stack, consisting of MongoDB, Express.js, React.js, and Node.js. The project provides a complete online platform for browsing books, managing shopping carts, maintaining wishlists, submitting reviews and ratings, and placing orders through a secure and user-friendly interface.

JWT authentication and bcrypt password encryption ensure secure access to protected resources while MongoDB provides efficient storage of application data. The React frontend delivers a responsive user interface, and Express.js REST APIs facilitate communication between the frontend and backend.

The application satisfies all the functional requirements identified during the analysis phase and demonstrates the practical implementation of modern full-stack web development technologies.

The modular architecture adopted in this project allows future enhancements without major modifications to the existing codebase. Overall, the Book Store application provides a scalable, reliable, and efficient solution for online book purchasing and management.